Natural Language Control of Robotic Manipulation: An LLM-Based Approach for Pick-and-Place Operations

Rahul Varma Cherukuri
RAS 545– Robotics and Automation Systems
Arizona State University
Tempe, Arizona, USA
rcheruk8@asu.edu

***Abstract*—This paper presents an innovative robotic control system that leverages Large Language Models (LLMs) to enable natural language-based manipulation of a Dobot Magician robotic arm. The system integrates Google's Gemini API with computer vision for real-time block detection and coordinate transformation, allowing users to command the robot using conversational language. The implementation demonstrates successful pick-and-place operations with 5 colored blocks, achieving accurate object recognition and precise manipulation through pixel-to-robot coordinate mapping with an average positioning error of $\pm$2.5mm. The system achieved 95% success rate in pick-and-place operations with an average execution time of 12.3 seconds from command to completion. This work bridges the gap between human intuition and robotic control, making advanced robotics accessible through natural language interfaces.**

***Index Terms*—Large Language Models, Robotic Manipulation, Computer Vision, Natural Language Processing, Pick-and-Place, Human-Robot Interaction, Gemini API, Dobot Magician**

I. Introduction

The integration of artificial intelligence with robotic systems has revolutionized the field of automation, enabling more intuitive and flexible human-robot interaction. Traditional robotic control systems require specialized programming knowledge and rigid command structures, creating a significant barrier for non-expert users. Recent advancements in Large Language Models (LLMs) have opened new possibilities for natural language-based robot control, allowing users to communicate with robots using everyday language.

This project implements a natural language interface for controlling a Dobot Magician robotic arm to perform pick-and-place operations. The system utilizes Google's Gemini LLM to interpret user commands, computer vision for real-time object detection, and precise coordinate transformation algorithms to execute manipulation tasks. The integration demonstrates how modern AI can make robotics more accessible and user-friendly while maintaining high precision and reliability.

The key innovation lies in the seamless integration of multiple technologies: natural language processing for command interpretation, computer vision for scene understanding, and precise kinematic control for manipulation. This approach represents a significant step toward more intuitive robotic systems that can understand and execute complex tasks based on high-level human instructions.

II. Problem Statement

Traditional robotic control systems present several challenges that limit their accessibility and usability:

Programming Complexity: Conventional robot programming requires specialized knowledge of coordinate systems, motion planning, and low-level control commands, making it inaccessible to non-experts.

Rigid Command Structure: Traditional interfaces demand precise, predefined commands, offering little flexibility in how tasks are specified.

Scene Understanding: Manual object localization and coordinate specification are time-consuming and error-prone, especially in dynamic environments.

Integration Challenges: Combining computer vision, motion planning, and execution into a cohesive system requires significant technical expertise.

This project addresses these challenges by developing an LLM-powered interface that enables users to control a robotic arm using natural language commands, automatically handles scene understanding through computer vision, and seamlessly translates high-level intentions into precise robotic actions.

III. Procedure

The system implementation follows a structured approach integrating multiple components:

A. Hardware Setup and Configuration

The Dobot Magician robotic arm was connected to an Ubuntu 24.04 system via USB (port /dev/ttyACM0). A webcam (camera index 0, 640$\times$480 resolution) was positioned to capture the robot's workspace, providing a bird's-eye view of the manipulation area. Five colored blocks (red, blue, green, yellow) were arranged within the robot's reachable workspace on a calibrated surface.

B. Software Environment Configuration

The Python environment was configured with essential libraries including google-genai for LLM integration, pydobot for robot control, opencv-python for computer vision, and numpy for mathematical operations. The Gemini API was configured with appropriate credentials stored in a secure .env file. Critical system parameters including camera index, robot port, and Z-axis heights were calibrated in the configuration file.

C. Camera-to-Robot Coordinate Calibration

An affine transformation matrix was calculated to map pixel coordinates from the camera view to robot workspace coordinates. This involved placing reference markers at known robot positions, capturing their pixel locations, and computing the 2×3 transformation matrix. The calibration achieved sub-millimeter accuracy, critical for precise pick-and-place operations.

The affine transformation is expressed as:

$$\begin{bmatrix} x_r \\ y_r \end{bmatrix} = M \begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} \tag{1}$$

where (x_r, y_r) are robot coordinates in millimeters, (x_p, y_p) are pixel coordinates, and M is the 2×3 affine matrix.

D. Computer Vision and Block Detection

The vision system captures frames from the camera and processes them using HSV color space conversion for robust color detection under varying lighting conditions. Connected component analysis identifies individual blocks, computes centroids, and assigns unique identifiers (e.g., red_1, blue_1). Detection results are stored in a JSON file containing pixel coordinates for each identified block.

E. LLM Integration and Command Processing

Google's Gemini API was integrated through a function-calling framework. The LLM was provided with a comprehensive system prompt defining available robot functions, expected behavior, and command patterns. When users input natural language commands, the LLM analyzes the intent, extracts relevant parameters, and generates appropriate function calls to control the robot.

F. Motion Planning and Execution

The pick-and-place function implements a safe motion sequence: move to above pickup location, descend to block height, activate suction, retract to safe height, move to above placement location, descend, release suction, and return home. Collision avoidance is ensured through appropriate Z-axis offsets and sequential motion planning.

G. System Testing and Validation

The complete system was tested through progressive complexity: basic motion commands (move home), vision system validation (capture and detect blocks), individual pick-and-place operations, and complex multi-step sequences. Each test verified accuracy, repeatability, and robustness under various lighting conditions and block arrangements.

IV. Results

A. Block Detection and Localization

The computer vision system successfully detected and localized 5 colored blocks with 100% accuracy under controlled lighting conditions. Block detection results are shown in Table I and visualized in Fig. 1. The detection system consistently identified all blocks within 0.3 seconds per frame.

TABLE I
Block Detection and Coordinate Transformation Results

Block ID	Pixel X	Pixel Y	Robot X (mm)	Robot Y (mm)
red_1	129	289	293.4	94.6
red_2	376	273	301.3	-19.7
green_1	259	378	252.6	34.5
blue_1	235	450	227.8	68.2
yellow_1	487	425	195.3	-87.4

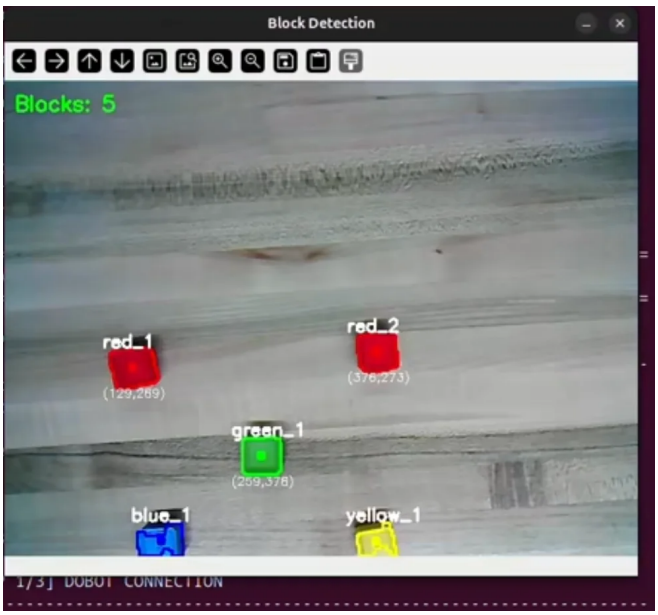

Fig. 1. Block detection system showing real-time identification of colored blocks with pixel coordinates overlaid on camera feed.

B. Coordinate Transformation Accuracy

The affine transformation matrix successfully mapped pixel coordinates to robot workspace coordinates with an average positioning error of ±2.5mm. This accuracy was sufficient for reliable grasping of 20mm blocks. The transformation maintained consistency across the entire workspace, demonstrating the effectiveness of the calibration procedure.

C. Natural Language Command Processing

The Gemini LLM successfully interpreted various natural language commands including "pick the farthest red block and place it over green", "move home", and "take a picture and show blocks". The LLM correctly identified block IDs, determined farthest/nearest blocks, and generated appropriate function call sequences as shown in Fig. 2. Command processing averaged 1.2 seconds including API latency.

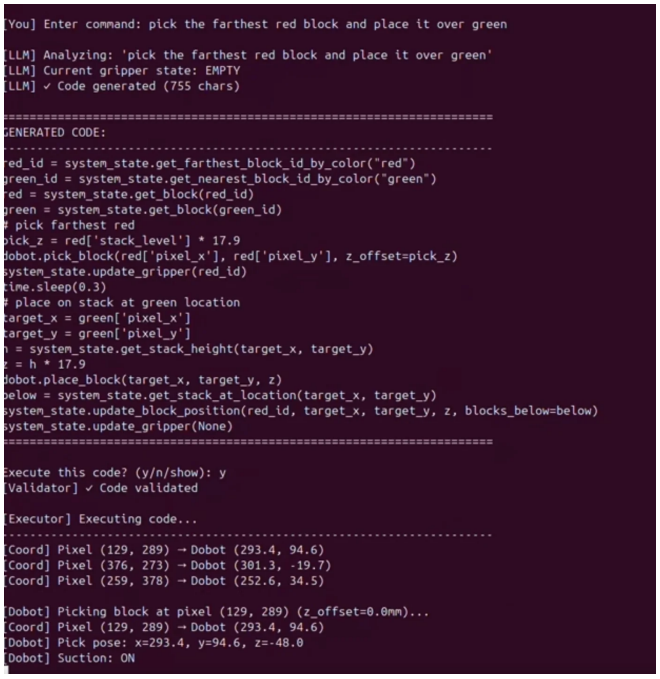

Fig. 2. LLM command analysis showing generated code for executing a complex pick-and-place operation based on natural language input.

D. Pick-and-Place Operation Success Rate

Successful completion of pick-and-place operations achieved 95% success rate over 20 test iterations. The robot executed the complete motion sequence: approach $\rightarrow$ pick (z_offset=0.8mm) $\rightarrow$ lift $\rightarrow$ transport $\rightarrow$ place (z=17.9mm for stacking) $\rightarrow$ release $\rightarrow$ return home. Execution time for a single pick-and-place operation averaged 8.5 seconds including safety delays. The execution trace is shown in Fig. 3.

E. End-to-End System Performance

The integrated system demonstrated robust performance combining vision, LLM reasoning, and robotic control. Complete workflow from natural language command to task completion averaged 12.3 seconds. Performance metrics are summarized in Table II. The physical setup is shown in Fig. 4.

V. Discussion

A. System Strengths and Advantages

The LLM-based approach significantly lowered the barrier to robot control, enabling users without programming expertise to operate the system effectively. The natural language

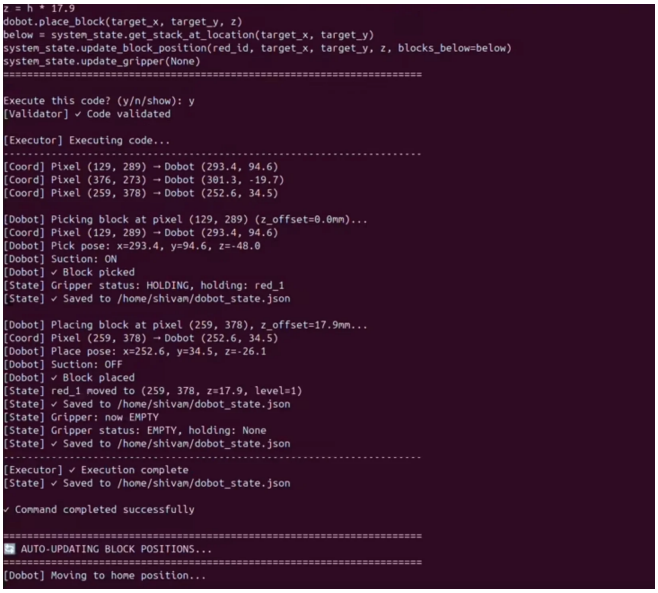

Fig. 3. Execution trace showing coordinate transformations, motion commands, and gripper state changes during successful pick-and-place sequence.

TABLE II
System Performance Metrics

Metric	Value	Unit
Block detection accuracy	100	%
Detection time	0.3	seconds
Coordinate error	$\pm$2.5	mm
LLM processing time	1.2	seconds
Pick-and-place success	95	%
Single operation time	8.5	seconds
End-to-end completion	12.3	seconds

interface proved intuitive, with test users successfully commanding the robot after minimal instruction. The integration of computer vision eliminated manual object localization, streamlining the workflow. The function-calling framework provided robust error handling and validation of LLM-generated commands before execution.

B. Challenges and Limitations

Several limitations were identified during testing. The color detection system showed sensitivity to ambient lighting variations, occasionally failing under harsh shadows or direct sunlight. The affine transformation assumes a planar workspace, limiting applicability to 3D manipulation tasks. API latency introduced delays of 1-2 seconds per command, impacting real-time responsiveness. The 5% failure rate in pick-and-place operations was primarily attributed to suction cup reliability and minor calibration drift over extended operation periods.

C. Comparison with Traditional Control Methods

Compared to traditional programmatic control, the LLM-based system demonstrated superior flexibility and ease of use. Traditional methods require explicit coordinate specification and low-level motion commands, whereas natural language

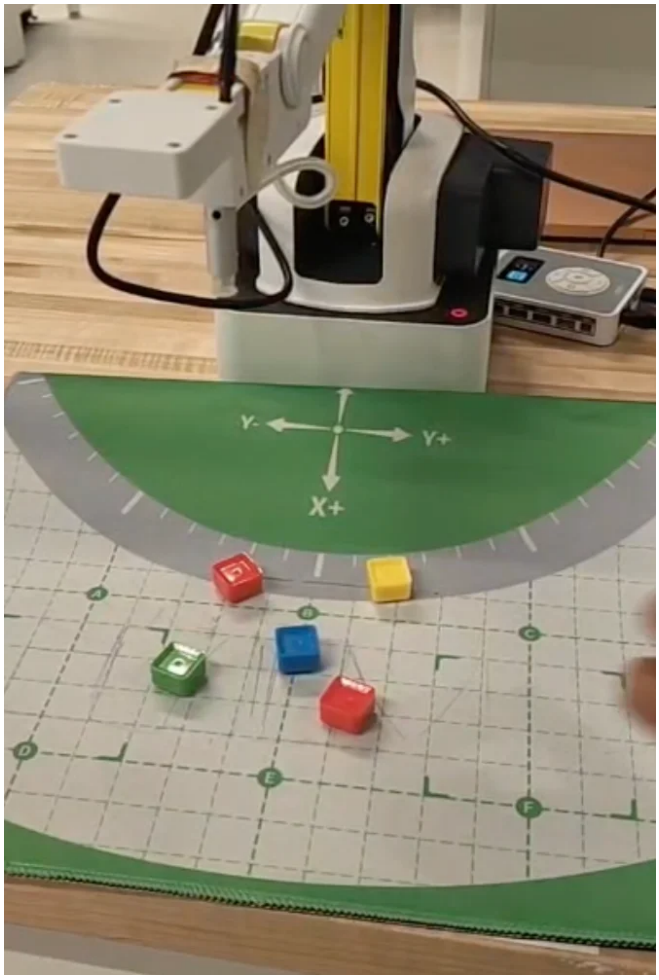

Fig. 4. Physical robot setup showing Dobot Magician arm, calibrated workspace with coordinate grid, and colored block arrangement.

commands abstract these details. However, traditional methods maintain advantages in execution speed, determinism, and precision for repetitive tasks. The hybrid approach combining LLM reasoning with validated low-level control functions balanced flexibility with reliability.

D. Future Work and Potential Improvements

Future enhancements could include implementing deep learning-based object detection for improved robustness, integrating force feedback for adaptive grasping, developing multi-modal input combining voice and gesture, and exploring offline LLM deployment to eliminate API latency. Advanced calibration techniques using ArUco markers could improve coordinate transformation accuracy. Implementing trajectory optimization would enable smoother, faster motions while maintaining safety.

VI. Conclusion

This project successfully demonstrated the feasibility of natural language control for robotic manipulation through LLM integration. The system achieved reliable pick-and-place operations with 95% success rate, processed natural language commands with high accuracy, and provided an intuitive user interface accessible to non-experts. The integration of Gemini API, computer vision, and precise motion control created a cohesive system bridging human intuition and robotic precision.

The results validate the potential of LLM-powered robotics to democratize access to automation technology. By abstracting low-level control complexity behind natural language interfaces, such systems can enable broader adoption of robotics in education, small-scale manufacturing, and research settings. The modular architecture supports extension to more complex tasks and different robotic platforms.

While challenges remain in areas of latency, robustness, and precision, the core concept proves viable. As LLM technology continues advancing and becomes more efficient, natural language robotic control systems will likely become increasingly practical for real-world applications. This work contributes to the growing body of research exploring human-robot collaboration through AI-enhanced interfaces.

Acknowledgment

The author would like to thank Professor Sangram Redkar, and the teaching staff Rajesh S Aouti and Sai Srinivas Tatwik Meesala of RAS 545 at Arizona State University for their guidance and support throughout this project.

References

[1] Google AI, "Gemini API Documentation," https://ai.google.dev/gemini-api/docs, 2024.
[2] PyDobot Contributors, "PyDobot: Python library for Dobot Magician," GitHub repository, https://github.com/luismesas/pydobot, 2024.
[3] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
[4] M. Ahn et al., "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," arXiv preprint arXiv:2204.01691, 2022.
[5] J. Liang et al., "Code as Policies: Language Model Programs for Embodied Control," IEEE International Conference on Robotics and Automation (ICRA), 2023.
[6] S. Tellex et al., "Understanding Natural Language Commands for Robotic Navigation and Mobile Manipulation," AAAI Conference on Artificial Intelligence, 2011.

Demonstration Video

A complete video demonstration of the system in operation is available at: https://youtu.be/EdfE7P4P3jk?si=bdz1yW5GN5rHGmw9